CS240 Assignment5

Author: Matthew Woo

Date: Oct 23, 2026

GitHub Repo: https://github.com/matthewoo527/cs240-assignment5/

Part 1: Assemble Instructions by Hand

```
Convert these MIPS instructions into 32-bit binary machine code by hand:
addi $t0, $zero, 3
addi $t1, $zero, 5
add  $t2, $t0, $t1
sub  $t3, $t1, $t0
For each instruction, identify its instruction format and show the fields used to
construct its binary encoding.
```

From ChatGPT, I got some notes:

Format

R Type (e.g. `add`, `sub`)

1. rs = first source
2. rt = second source
3. rd = destination

opcode	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

I Type (e.g. `addi`, `lw`, `sw`)

1. rs = first source
2. rt = destination

opcode	rs	rt	immediate
6 bits	5 bits	5 bits	16 bits

J Type (e.g. `j`, `jal`)

opcode	target address
6 bits	26 bits

Register binary reference

Register	Number	5-bit Binary
$zero	0	00000
$t0	8	01000
$t1	9	01001
$t2	10	01010
$t3	11	01011
$t4	12	01100
$t5	13	01101
$t6	14	01110
$t7	15	01111

Some MIPS Opcode Reference

Instruction	Format	Opcode (6 bits)	Funct (R-Type)
add	R	000000	100000
sub	R	000000	100010
and	R	000000	100100
or	R	000000	100101
slt	R	000000	101010
jr	R	000000	001000
addi	I	001000	—

Instruction	Format	Opcode (6 bits)	Funct (R-Type)
lw	I	100011	—
sw	I	101011	—
beq	I	000100	—
bne	I	000101	—
j	J	000010	—
jal	J	000011	—

Converting

`addi $t0, $zero, 3`
It is an I type.

opcode	rs	rt	immediate
addi	$zero	$t0	3
001000	00000	01000	0000000000000011

So the 32 bits machine code for `addi $t0, $zero, 3` is `00100000000010000000000000000011`
`addi $t1, $zero, 5`
It is an I type

opcode	rs	rt	immediate
addi	$zero	$t1	5
001000	00000	01001	0000000000000101

So the 32 bits machine code for `addi $t1, $zero, 5` is `00100000000010010000000000000101`
`add $t2, $t0, $t1`
It is an R type.

opcode	rs	rt	rd	shamt	funct
add	$t0	$t1	$t2	No shift amount	funct of add
000000	01000	01001	01010	00000	100000

So the 32 bits machine code for `add $t2, $t0, $t1` is `00000001000010010101000000100000`
`sub $t3, $t1, $t0`
It is an R type.

opcode	rs	rt	rd	shamt	funct
add	$t1	$t0	$t3	No shift amount	funct of sub
000000	01001	01000	01011	00000	100010

So the 32 bits machine code for `sub $t3, $t1, $t0` is `00000001001010000101100000100010`

Part 2: Represent Machine Code as Binary Bytes

Using the machine code from Part 1:

1. Show each 32-bit instruction as four 8-bit bytes.

Instruction 1: `addi $t0, $zero, 3`
32 bits machine code for `addi $t0, $zero, 3` is `00100000000010000000000000000011`

Byte 1	Byte 2	Byte 3	Byte 4
00100000	00001000	00000000	00000011

Instruction 2: `addi $t1, $zero, 5`
32 bits machine code for `addi $t1, $zero, 5` is `00100000000010010000000000000101`

Byte 1	Byte 2	Byte 3	Byte 4
00100000	00001001	00000000	00000101

Instruction 3: `add $t2, $t0, $t1`
32 bits machine code for `add $t2, $t0, $t1` is `00000001000010010101000000100000`

Byte 1	Byte 2	Byte 3	Byte 4
00000001	00001001	01010000	00100000

Instruction 4: `sub $t3, $t1, $t0`

32 bits machine code for `sub $t3, $t1, $t0` is `00000001001010000101100000100010`

Byte 1	Byte 2	Byte 3	Byte 4
00000001	00101000	01011000	00100010

2. Show each byte in hexadecimal.

Instruction 1: `addi $t0, $zero, 3`
32 bits machine code for `addi $t0, $zero, 3` is `00100000000010000000000000000011`

	Byte 1	Byte 2	Byte 3	Byte 4
Binary	00100000	00001000	00000000	00000011
Hexadecimal	20	08	00	03

Hexadecimal: 0x20080003

Instruction 2: `addi $t1, $zero, 5`
32 bits machine code for `addi $t1, $zero, 5` is `00100000000010010000000000000101`

	Byte 1	Byte 2	Byte 3	Byte 4
Binary	00100000	00001001	00000000	00000101
Hexadecimal	20	09	00	05

Hexadecimal: 0x20090005

Instruction 3: `add $t2, $t0, $t1`
32 bits machine code for `add $t2, $t0, $t1` is `00000001000010010101000000100000`

	Byte 1	Byte 2	Byte 3	Byte 4
Binary	00000001	00001001	01010000	00100000
Hexadecimal	01	09	50	20

Hexadecimal: 0x01095020

Instruction 4: `sub $t3, $t1, $t0`
32 bits machine code for `sub $t3, $t1, $t0` is `00000001001010000101100000100010`

	Byte 1	Byte 2	Byte 3	Byte 4
Binary	00000001	00101000	01011000	00100010
Hexadecimal	01	28	58	22

Hexadecimal: 0x01285822

3. Explain how the two output formats differ:

`mhc-ascii`: A text file containing readable 0 and 1 characters.
`mhc-bin`: A binary file containing the actual machine-code bytes.
A 32-bit instruction requires 32 text characters in `mhc-ascii`, excluding spaces and line breaks, but only four bytes in `mhc-bin`.
Explanation:

`mhc-ascii` only stores `0` and `1`. Since each MIPS instruction is 32 bits, it will have 32 `0`s and `1`s per instruction and 32 text characters for each instruction.
`mhc-bin` stores the actual machine-code bytes instead of text characters. It stores every 8 bits as 1 byte, so it will only need 4 bytes for a 32-bit instruction. We can use hexadecimal to represent these bytes.

Part 3: Write an Assembler

AI Disclosure: I used ChatGPT 6 to learn how to change the professor's code to read an input `fizzbuzz.asm` file and output `fizzbuzz.mhc-ascii` and `fizzbuzz.mhc-bin`files. The base/original code is from the professor.

How to run?

Put the fizzbizz.asm program in the same folder as `Assembler.java`, make sure the file name of fizzbuzz.asm is the same as the file name stated in the Assembler.java. In my case, I used the name ```Woo_Matthew_Old_FizzBuzz.asm`

Source Code

Assembler.java

```
import java.io.DataOutputStream;
import java.io.File;
import java.io.FileNotFoundException;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
```

```java
import java.util.Scanner;
import java.util.Set;

/*
 * Simple two-pass MIPS assembler (Java 8+).
 * Place Assembler.java and FizzBuzz.asm in the same folder.
 * Run: javac Assembler.java
 *      java Assembler
 *
 * Output: fizzbuzz.mhc-ascii and fizzbuzz.mhc-bin
 * .text starts at 0x00400000; .data starts at 0x10010000.
 * Words are stored BIG-ENDIAN. No automatic branch delay slots.
 *
 * Instructions: addi, addiu, addu, ori, lui, div, mfhi, sll,
 *               beq, bne, j, jal, jr, lw, sw, syscall
 * Pseudoinstructions: li, la, move, rem, nop
 * Directives: .text, .data, .asciiz, .globl, .global
 *
 * Binary format (custom, not ELF):
 * 4 bytes "MHC1" | text base (4) | text length (4) |
 * data base (4) | data length (4) | .text bytes | .data bytes
 * All 4-byte numbers are big-endian.
 */
public class Assembler {
    static final int TEXT_BASE = 0x00400000;
    static final int DATA_BASE = 0x10010000;

    private static final String[] REGISTER_NAMES = {
        "zero", "at", "v0", "v1", "a0", "a1", "a2", "a3",
        "t0", "t1", "t2", "t3", "t4", "t5", "t6", "t7",
        "s0", "s1", "s2", "s3", "s4", "s5", "s6", "s7",
        "t8", "t9", "k0", "k1", "gp", "sp", "fp", "ra"
    };

    static class Instruction {
        int line, address;
        String text;
        Instruction(int line, int address, String text) {
            this.line = line;
            this.address = address;
            this.text = text;
        }
    }

    static class Result {
        byte[] text, data;
        Result(byte[] text, byte[] data) {
            this.text = text;
            this.data = data;
        }
    }

    private final Map<String, Integer> labels = new HashMap<>();
    private final Set<String> textLabels = new HashSet<>();
    private final List<String> globals = new ArrayList<>();
    private final List<Instruction> instructions = new ArrayList<>();
    private final List<Byte> data = new ArrayList<>();
    private boolean inData = false;
    private int pc = TEXT_BASE;

    public static void main(String[] args) {
```

```
        try {
            // 1. Read FizzBuzz.asm from the current folder using Scanner.
            StringBuilder source = new StringBuilder();
            try (Scanner input = new Scanner(new File("Woo_Matthew_Old_FizzBuzz.asm"),
"UTF-8")) {
                while (input.hasNextLine()) {
                    source.append(input.nextLine()).append('\n');
                }
            }

            // 2. Assemble the source into instruction bytes and data bytes.
            Result result = assemble(source.toString());

            // 3. Save both formats.
            writeAscii(result);
            writeBinary(result);
            System.out.println("Created fizzbuzz.mhc-ascii");
            System.out.println("Created fizzbuzz.mhc-bin");
        } catch (FileNotFoundException e) {
            System.err.println("Cannot find FizzBuzz.asm in this folder.");
        } catch (IOException | IllegalArgumentException e) {
            System.err.println("Assembly failed: " + e.getMessage());
        }
    }

    // Text file: 32 visible 0/1 characters per instruction.
    // Data is labeled separately and stored as 8 bits per byte.
    static void writeAscii(Result result) throws IOException {
        try (PrintWriter out = new PrintWriter("fizzbuzz.mhc-ascii", "UTF-8")) {
            out.println("# .text base=0x00400000 bytes=" + result.text.length);
            for (int i = 0; i < result.text.length; i += 4) {
                for (int j = 0; j < 4; j++) {
                    out.print(eightBits(result.text[i + j]));
                }
                out.println();
            }
            out.println("# .data base=0x10010000 bytes=" + result.data.length);
            for (byte b : result.data) {
                out.println(eightBits(b));
            }
            if (out.checkError()) throw new IOException("Could not write ASCII file.");
        }
    }

    static String eightBits(byte value) {
        String bits = Integer.toBinaryString(value & 255);
        return "00000000".substring(bits.length()) + bits;
    }

    // Binary file: 20-byte header followed by actual .text and .data bytes.
    static void writeBinary(Result result) throws IOException {
        try (DataOutputStream out = new DataOutputStream(new
FileOutputStream("fizzbuzz.mhc-bin"))) {
            out.writeBytes("MHC1");
            out.writeInt(TEXT_BASE);
            out.writeInt(result.text.length);
            out.writeInt(DATA_BASE);
            out.writeInt(result.data.length);
            out.write(result.text);
            out.write(result.data);
        }
```

```
    }

    static Result assemble(String source) {
        Assembler assembler = new Assembler();
        assembler.firstPass(source);
        if (assembler.instructions.isEmpty()) fail("No instructions found.");
        for (String name : assembler.globals) assembler.label(name);

        // Pass 2: encode instructions after labels are known.
        byte[] text = new byte[assembler.instructions.size() * 4];
        int index = 0;
        for (Instruction instruction : assembler.instructions) {
            try {
                int word = assembler.encode(instruction);
                for (int shift = 24; shift >= 0; shift -= 8) {
                    text[index++] = (byte) (word >>> shift);
                }
            } catch (IllegalArgumentException e) {
                throw new IllegalArgumentException("Line " + instruction.line + ": " +
e.getMessage());
            }
        }
        byte[] bytes = new byte[assembler.data.size()];
        for (int i = 0; i < bytes.length; i++) bytes[i] = assembler.data.get(i);
        return new Result(text, bytes);
    }

    // Pass 1: expand pseudos BEFORE calculating following label addresses.
    private void firstPass(String source) {
        if (source.startsWith("\uFEFF")) source = source.substring(1);
        String[] lines = source.split("\\r\\n|\\r|\\n");
        for (int i = 0; i < lines.length; i++) {
            try {
                String text = withoutComment(lines[i]).trim();
                while (true) {
                    int colon = text.indexOf(':');
                    if (colon < 0) break;
                    String name = text.substring(0, colon).trim();
                    if (!name.matches("[A-Za-z_][A-Za-z0-9_]*")) break;
                    if (labels.containsKey(name)) fail("Duplicate label: " + name);
                    labels.put(name, inData ? DATA_BASE + data.size() : pc);
                    if (!inData) textLabels.add(name);
                    text = text.substring(colon + 1).trim();
                }
                if (text.isEmpty()) continue;
                if (text.startsWith(".")) {
                    directive(text);
                } else {
                    if (inData) fail("Instructions must be in .text.");
                    expand(text, i + 1);
                }
            } catch (IllegalArgumentException e) {
                throw new IllegalArgumentException("Line " + (i + 1) + ": " +
e.getMessage());
            }
        }
    }

    private void directive(String text) {
        String[] p = text.split("\\s+", 2);
        String value = p.length == 2 ? p[1].trim() : "";
```

```
        switch (p[0]) {
            case ".text":
            case ".data":
                boolean nextData = p[0].equals(".data");
                int base = nextData ? DATA_BASE : TEXT_BASE;
                if (!value.isEmpty()) {
                    if (number(value) != base) fail("Only the default segment base is
supported.");
                    if (nextData ? !data.isEmpty() : !instructions.isEmpty()) {
                        fail("Explicit segment address must precede its contents.");
                    }
                }
                inData = nextData;
                break;
            case ".globl":
            case ".global":
                if (!value.matches("[A-Za-z_][A-Za-z0-9_]*")) fail("Expected one global
label.");
                globals.add(value);
                break;
            case ".asciiz":
                if (!inData) fail(".asciiz must be in .data.");
                String decoded = quoted(value, '"');
                for (int i = 0; i < decoded.length(); i++) data.add((byte)
decoded.charAt(i));
                data.add((byte) 0);
                break;
            default:
                fail("Unsupported directive: " + p[0]);
        }
    }

    private void emit(String text, int line) {
        instructions.add(new Instruction(line, pc, text));
        pc += 4;
    }

    private void loadImmediate(String register, int value, int line) {
        reg(register);
        if (value >= -32768 && value <= 32767) {
            emit("addiu " + register + ", $zero, " + value, line);
        } else if (value >= 0 && value <= 65535) {
            emit("ori " + register + ", $zero, " + value, line);
        } else {
            emit("lui " + register + ", " + (value >>> 16), line);
            emit("ori " + register + ", " + register + ", " + (value & 65535), line);
        }
    }

    private void expand(String text, int line) {
        String[] p = text.split("\\s+", 2);
        String op = lowerAscii(p[0]);
        String[] a = operands(p);
        switch (op) {
            case "li":
                count(a, 2);
                loadImmediate(a[0], number(a[1]), line);
                break;
            case "la":
                count(a, 2);
                reg(a[0]);
```

```
            if (!a[1].matches("[A-Za-z_][A-Za-z0-9_]*")) fail("la expects a label.");
            emit("lui " + a[0] + ", hi(" + a[1] + ")", line);
            emit("ori " + a[0] + ", " + a[0] + ", lo(" + a[1] + ")", line);
            break;
        case "move":
            count(a, 2);
            emit("addu " + a[0] + ", " + a[1] + ", $zero", line);
            break;
        case "rem":
            count(a, 3);
            if (reg(a[0]) == 1 || reg(a[1]) == 1) fail("rem reserves $at.");
            int divisor = number(a[2]);
            if (divisor == 0) fail("rem divisor cannot be zero.");
            loadImmediate("$at", divisor, line);
            emit("div " + a[1] + ", $at", line);
            emit("mfhi " + a[0], line);
            break;
        case "nop":
            count(a, 0);
            emit("sll $zero, $zero, 0", line);
            break;
        default:
            emit(text, line);
    }
}

private int encode(Instruction instruction) {
    String[] p = instruction.text.split("\\s+", 2);
    String op = lowerAscii(p[0]);
    String[] a = operands(p);
    switch (op) {
        case "addi":
        case "addiu":
        case "ori":
            count(a, 3);
            int immediate = value(a[2]);
            range(immediate, op.equals("ori") ? 0 : -32768,
                             op.equals("ori") ? 65535 : 32767);
            return iType(op.equals("addi") ? 8 : op.equals("addiu") ? 9 : 13,
                         reg(a[1]), reg(a[0]), immediate);
        case "lui":
            count(a, 2);
            return iType(15, 0, reg(a[0]), range(value(a[1]), 0, 65535));
        case "addu":
            count(a, 3);
            return rType(reg(a[1]), reg(a[2]), reg(a[0]), 0, 0x21);
        case "div":
            count(a, 2);
            return rType(reg(a[0]), reg(a[1]), 0, 0, 0x1A);
        case "mfhi":
            count(a, 1);
            return rType(0, 0, reg(a[0]), 0, 0x10);
        case "sll":
            count(a, 3);
            return rType(0, reg(a[1]), reg(a[0]), range(number(a[2]), 0, 31), 0);
        case "beq":
        case "bne":
            count(a, 3);
            int offset = (codeLabel(a[2]) - (instruction.address + 4)) / 4;
            return iType(op.equals("beq") ? 4 : 5, reg(a[0]), reg(a[1]),
                         range(offset, -32768, 32767));
```

```
            case "j":
            case "jal":
                count(a, 1);
                int target = codeLabel(a[0]);
                if ((target & 0xF0000000) != ((instruction.address + 4) & 0xF0000000)) {
                    fail("Jump target is out of range.");
                }
                return ((op.equals("j") ? 2 : 3) << 26) | ((target >>> 2) & 0x03FFFFFF);
            case "jr":
                count(a, 1);
                return rType(reg(a[0]), 0, 0, 0, 8);
            case "lw":
            case "sw":
                count(a, 2);
                String memory = a[1].replaceAll("\\s+", "");
                if (!memory.matches("[^()]+\\(\\$[A-Za-z0-9]+\\)")) {
                    fail("Expected offset($register): " + a[1]);
                }
                int left = memory.indexOf('(');
                int base = reg(memory.substring(left + 1, memory.length() - 1));
                int displacement = range(number(memory.substring(0, left)), -32768,
32767);
                return iType(op.equals("lw") ? 35 : 43, base, reg(a[0]), displacement);
            case "syscall":
                count(a, 0);
                return 0x0000000C;
            default:
                throw new IllegalArgumentException("Unsupported instruction: " + op);
        }
    }

    // R: opcode(6), rs(5), rt(5), rd(5), shamt(5), funct(6).
    private static int rType(int rs, int rt, int rd, int shift, int function) {
        return (rs << 21) | (rt << 16) | (rd << 11) | (shift << 6) | function;
    }

    // I: opcode(6), rs(5), rt(5), immediate(16).
    private static int iType(int opcode, int rs, int rt, int immediate) {
        return (opcode << 26) | (rs << 21) | (rt << 16) | (immediate & 0xFFFF);
    }

    private int label(String name) {
        Integer result = labels.get(name);
        if (result == null) fail("Undefined label: " + name);
        return result;
    }

    private int codeLabel(String name) {
        int address = label(name);
        if (!textLabels.contains(name)) fail("Branch/jump target is not in .text: " +
name);
        return address;
    }

    private int value(String text) {
        if (text.startsWith("hi(") && text.endsWith(")")) {
            return label(text.substring(3, text.length() - 1)) >>> 16;
        }
        if (text.startsWith("lo(") && text.endsWith(")")) {
            return label(text.substring(3, text.length() - 1)) & 65535;
        }
```

```
        return number(text);
    }

    private static int reg(String name) {
        name = lowerAscii(name.trim());
        if (name.equals("$s8")) return 30;
        for (int i = 0; i < REGISTER_NAMES.length; i++) {
            if (name.equals("$" + REGISTER_NAMES[i]) || name.equals("$" + i)) return i;
        }
        throw new IllegalArgumentException("Unknown register: " + name);
    }

    private static int number(String text) {
        text = text.trim();
        if (text.startsWith("'")) {
            String character = quoted(text, '\'');
            if (character.length() != 1) fail("Expected one character: " + text);
            return character.charAt(0);
        }
        try {
            boolean negative = text.startsWith("-");
            String digits = text;
            if (digits.startsWith("-") || digits.startsWith("+")) digits =
digits.substring(1);
            int radix = 10;
            if (digits.startsWith("0x") || digits.startsWith("0X")) {
                radix = 16;
                digits = digits.substring(2);
            } else if (digits.startsWith("0b") || digits.startsWith("0B")) {
                radix = 2;
                digits = digits.substring(2);
            }
            if (digits.isEmpty() || digits.startsWith("-") || digits.startsWith("+")) {
                fail("Invalid integer: " + text);
            }
            long result = Long.parseLong(digits, radix);
            if (negative) result = -result;
            if (result < Integer.MIN_VALUE || result > 0xFFFFFFFFL) fail("Integer too
large: " + text);
            return (int) result;
        } catch (NumberFormatException e) {
            throw new IllegalArgumentException("Invalid integer: " + text);
        }
    }

    // Keep # inside quoted strings or character literals.
    private static String withoutComment(String text) {
        char quote = 0;
        for (int i = 0; i < text.length(); i++) {
            char c = text.charAt(i);
            if (quote != 0) {
                if (c == '\\') i++;
                else if (c == quote) quote = 0;
            } else if (c == '"' || c == '\'') {
                quote = c;
            } else if (c == '#') {
                return text.substring(0, i);
            }
        }
        if (quote != 0) fail("Unclosed quote.");
        return text;
```

```
    }

    private static String quoted(String text, char quote) {
        if (text.length() < 2 || text.charAt(0) != quote
                || text.charAt(text.length() - 1) != quote) fail("Expected quoted
literal: " + text);
        StringBuilder decoded = new StringBuilder();
        for (int i = 1; i < text.length() - 1; i++) {
            char c = text.charAt(i);
            if (c == quote) fail("Unexpected quote: " + text);
            if (c == '\\') {
                if (++i >= text.length() - 1) fail("Incomplete escape.");
                switch (text.charAt(i)) {
                    case 'n': c = '\n'; break;
                    case 'r': c = '\r'; break;
                    case 't': c = '\t'; break;
                    case '0': c = 0; break;
                    case '\\': c = '\\'; break;
                    case '\'': c = '\''; break;
                    case '"': c = '"'; break;
                    default: fail("Unsupported escape: \\" + text.charAt(i));
                }
            }
            if (c > 127) fail("This toy assembler accepts ASCII literals only.");
            decoded.append(c);
        }
        return decoded.toString();
    }

    private static String[] operands(String[] p) {
        if (p.length == 1 || p[1].trim().isEmpty()) return new String[0];
        // Split commas outside quotes so li $a0, ',' also works.
        List<String> result = new ArrayList<>();
        char quote = 0;
        int start = 0;
        for (int i = 0; i < p[1].length(); i++) {
            char c = p[1].charAt(i);
            if (quote != 0) {
                if (c == '\\') i++;
                else if (c == quote) quote = 0;
            } else if (c == '\'' || c == '"') {
                quote = c;
            } else if (c == ',') {
                result.add(p[1].substring(start, i).trim());
                start = i + 1;
            }
        }
        result.add(p[1].substring(start).trim());
        return result.toArray(new String[0]);
    }

    private static void count(String[] a, int expected) {
        if (a.length != expected) fail("Expected " + expected + " comma-separated
operand(s).");
        for (String operand : a) if (operand.isEmpty()) fail("Empty operand.");
    }

    private static int range(int value, int min, int max) {
        if (value < min || value > max) fail("Value out of range: " + value);
        return value;
    }
```

```
    private static void fail(String message) {
        throw new IllegalArgumentException(message);
    }

    private static String lowerAscii(String text) {
        return text.toLowerCase(java.util.Locale.ROOT);
    }
}
```

Input File: My FizzBuzz program from the last assignment

Woo_Matthew_Old_FizzBuzz.asm

```
li $t0, 1
li $t1, 101

loop:
j PrintItem
PrintItemEnd:
#increment
addi $t0, $t0, 1
bne $t0, $t1, loop

j Exit

PrintItem:
rem $t2, $t0, 3 #Save remainder to $t2, $t2 == $t0 & 3
rem $t3, $t0, 5 #Save remainder to $t3, $t3 == $t0 & 5
#Check 3
beq $t2, $zero, CheckFive  #if 3 pass(can divide by 3), then check if 5 pass(can divide
by 5)
#Check 5
beq $t3, $zero, PrintBuzz
#If not dividable by 3 or 5, then PrintInt
j PrintInt
PrintIntEnd:
#Check 5 if Check 3 Pass
CheckFive:
       beq $t3, $zero, PrintFizzBuzz #If yes then PrintFizzBuzz
       j PrintFizz #If not then PrintFizz because it checked 3 and passed
#print new line
# j PrintNewLine
PrintNewLineEnd:
j PrintItemEnd

PrintInt:
#print integer
li $v0, 1
move $a0, $t0
syscall

PrintNewLine:
li $v0, 11
li $a0, '\n'
syscall
j PrintNewLineEnd

PrintFizz:
.data
```

```
        Fizz: .asciiz "Fizz"
.text
li $v0, 4
la $a0, Fizz
syscall
j PrintFizzEnd
PrintFizzEnd:
j PrintNewLine

PrintBuzz:
.data
        Buzz: .asciiz "Buzz"
.text
li $v0, 4
la $a0, Buzz
syscall
j PrintBuzzEnd
PrintBuzzEnd:
j PrintNewLine

PrintFizzBuzz:
.data
        FizzBuzz: .asciiz "FizzBuzz"
.text
li $v0, 4
la $a0, FizzBuzz
syscall
j PrintFizzBuzzEnd
PrintFizzBuzzEnd:
j PrintNewLine

Exit:
    li $v0, 10
    syscall
```

Output File

fizzbuzz.mhc-ascii

```
# .text base=0x00400000 bytes=180
00100100000010000000000000000001
00100100000010010000000001100101
00001000000100000000000000000110
00100001000010000000000000000001
00010101000010011111111111111101
00001000000100000000000000101011
00100100000000010000000000000011
00000001000000010000000000011010
00000000000000000101000000010000
00100100000000010000000000000101
00000001000000010000000000011010
00000000000000000101100000010000
00010001010000000000000000000010
00010001011000000000000000010001
00001000000100000000000000010010
00010001011000000000000000010101
00001000000100000000000000011001
00001000000100000000000000000011
00100100000000010000000000000001
```

```
00000001000000000010000000100001
00000000000000000000000000001100
00100100000000100000000000001011
00100100000001000000000000001010
00000000000000000000000000001100
00001000000100000000000000010001
00100100000000100000000000000100
00111100000001000001000000000001
00110100100001000000000000000000
00000000000000000000000000001100
00001000000100000000000000011110
00001000000100000000000000010101
00100100000000100000000000000100
00111100000001000001000000000001
00110100100001000000000000000101
00000000000000000000000000001100
00001000000100000000000000100100
00001000000100000000000000010101
00100100000000100000000000000100
00111100000001000001000000000001
00110100100001000000000000001010
00000000000000000000000000001100
00001000000100000000000000101010
00001000000100000000000000010101
00100100000000100000000000001010
00000000000000000000000000001100
# .data base=0x10010000 bytes=19
01000110
01101001
01111010
01111010
00000000
01000010
01110101
01111010
01111010
00000000
01000110
01101001
01111010
01111010
01000010
01110101
01111010
01111010
00000000
```

fizzbuzz.mhc-ascii Screenshot

fizzbuzz.mhc-ascii — Edited

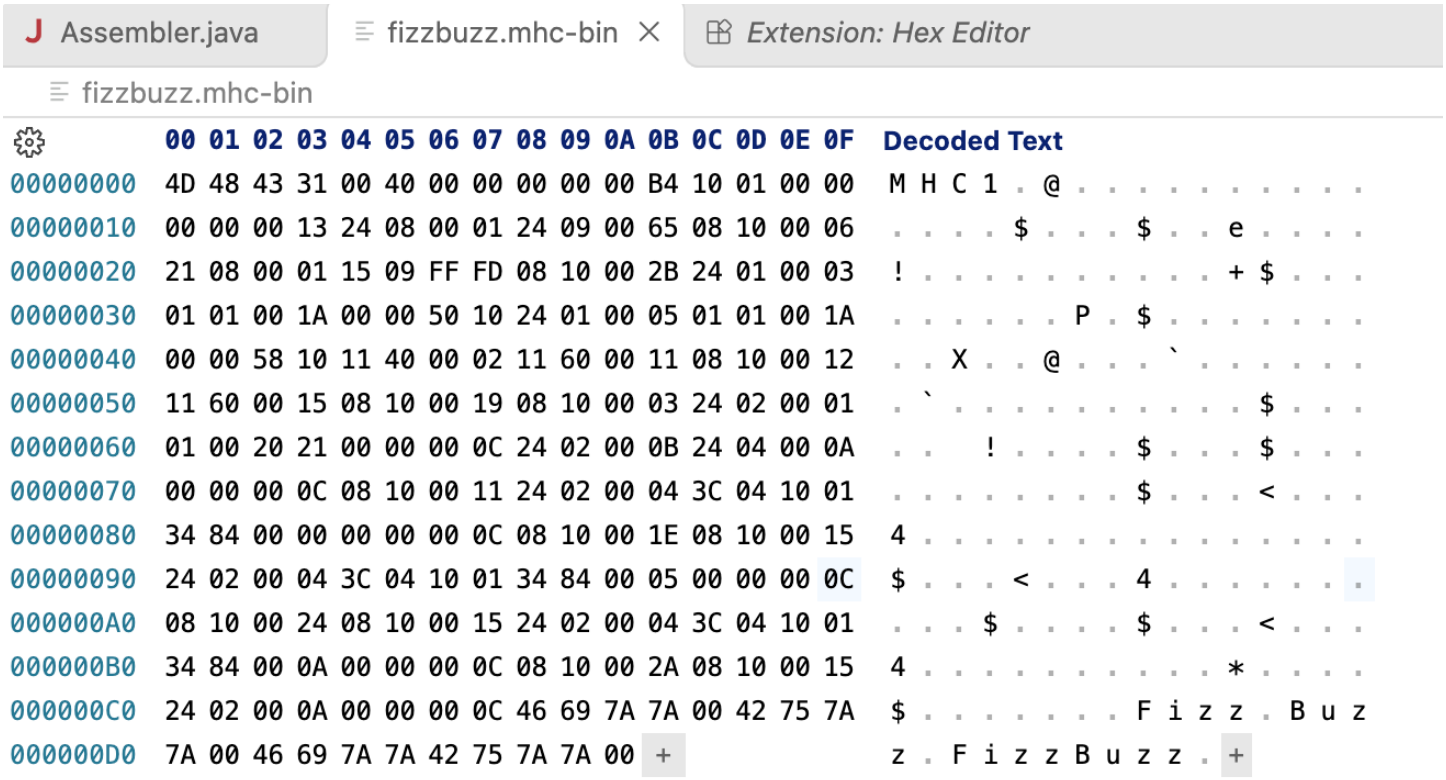
```
# .text base=0x00400000 bytes=180
00100100000010000000000000000001
00100100000010010000000001100101
00001000000100000000000000000110
00100001000010000000000000000001
00010101000010011111111111111101
00001000000100000000000000101011
00100100000000010000000000000011
00000001000000010000000000011010
00000000000000000101000000010000
00100100000000010000000000000101
00000001000000010000000000011010
00000000000000000101100000010000
00010001010000000000000000000010
00010001011000000000000000010001
00001000000100000000000000010010
00010001011000000000000000010101
00001000000100000000000000011001
00001000000100000000000000000011
00100100000000010000000000000001
00000001000000000010000000100001
00000000000000000000000000001100
00100100000000010000000000001011
00100100000001000000000000001010
00000000000000000000000000001100
00001000000100000000000000010001
00100100000000010000000000000100
00111100000001000001000000000001
00110100100001000000000000000000
00000000000000000000000000001100
00001000000100000000000000011110
00001000000100000000000000010101
00100100000000010000000000000100
00111100000001000001000000000001
00110100100001000000000000000101
00000000000000000000000000001100
00001000000100000000000000100100
00001000000100000000000000010101
00100100000000010000000000000100
00111100000001000001000000000001
00110100100001000000000000001010
00000000000000000000000000001100
00001000000100000000000000101010
00001000000100000000000000010101
00100100000000010000000000001010
00000000000000000000000000001100
# .data base=0x10010000 bytes=19
01000110
01101001
01111010
01111010
00000000
01000010
01110101
01111010
01111010
00000000
01000110
01101001
01111010
01111010
01000010
01110101
01111010
01111010
00000000
```

[fizzbuzz.mhc-bin]

I used Hex Editior extension on VS code since normal text editior cannot open a binary file.

```
00000000  4d 48 43 31 00 40 00 00  00 00 00 b4 10 01 00 00  |MHC1.@..........|
00000010  00 00 00 13 24 08 00 01  24 09 00 65 08 10 00 06  |....$...$..e....|
00000020  21 08 00 01 15 09 ff fd  08 10 00 2b 24 01 00 03  |!..........+$...|
00000030  01 01 00 1a 00 00 50 10  24 01 00 05 01 01 00 1a  |......P.$.......|
00000040  00 00 58 10 11 40 00 02  11 60 00 11 08 10 00 12  |..X..@...`......|
00000050  11 60 00 15 08 10 00 19  08 10 00 03 24 02 00 01  |.`..........$...|
00000060  01 00 20 21 00 00 00 0c  24 02 00 0b 24 04 00 0a  |.. !....$...$...|
00000070  00 00 00 0c 08 10 00 11  24 02 00 04 3c 04 10 01  |........$...<...|
00000080  34 84 00 00 00 00 00 0c  08 10 00 1e 08 10 00 15  |4...............|
00000090  24 02 00 04 3c 04 10 01  34 84 00 05 00 00 00 0c  |$...<...4.......|
000000a0  08 10 00 24 08 10 00 15  24 02 00 04 3c 04 10 01  |...$....$...<...|
000000b0  34 84 00 0a 00 00 00 0c  08 10 00 2a 08 10 00 15  |4..........*....|
000000c0  24 02 00 0a 00 00 00 0c  46 69 7a 7a 00 42 75 7a  |$.......Fizz.Buz|
000000d0  7a 00 46 69 7a 7a 42 75  7a 7a 00                 |z.FizzBuzz.|
000000db
```

fizzbuzz.mhc-bin Screenshot



Explaination

1. How It Works

The assembler is written in Java. It uses Scanner to read my FizzBuzz ASM file and converts the instructions into 32-bit machine code. It uses two passes: the first pass finds labels and their addresses, and the second pass converts the instructions into machine code. It then creates two output files.

2. Supported Features

The assembler supports instructions such as `addi`, `beq`, `bne`, `j`, and `syscall`. It also supports pseudoinstructions like `li`, `la`, `move`, and `rem`. It can handle labels and data directives such as .text, .data, and .asciiz.

3. File Layout

- mhc-ascii: Stores machine code as readable 0s and 1s, with 32 bits per instruction. Instructions and data are labeled separately.
- mhc-bin: Stores actual machine-code bytes, with 4 bytes per instruction. It has a 20-byte header, followed by the instructions and data.

The FizzBuzz binary file contains 180 bytes of instructions, 19 bytes of data, and a 20-byte header, totaling 219 bytes.

4. Starting Addresses

.text = 0x00400000 stores the instructions.

.data = 0x10010000 stores the strings and other data.

The assembler uses these addresses to calculate label locations.

5. Byte Order

The assembler uses big-endian, meaning the most significant byte comes first. For example, 0x24080001 is stored as 24 08 00 01.

6. Error Handling

The assembler displays error messages for unsupported instructions, invalid registers, missing labels, and other invalid inputs. It also reports when the ASM file cannot be found.

7. Testing

I tested my assembler using my FizzBuzz ASM program. It successfully generated both output files. I opened mhc-ascii in a text editor and used Hex Editior extension in VS Code to view mhc-bin in hexadecimal.

Part 4: Write a Disassembler

AI Disclosure: I used ChatGPT 6 to learn how to build a disassembler. It also helped me to build a disassembler, and I understand the code by read and type the code that it produces.

How to run?

This program will produce a file called `Reconstructed.asm` no matter what input it was. Make sure to rename the output file when it run the second time because the new output file will have the same file name and will replace the old file.

I first run `java Disassembler fizzbuzz.mhc-ascii` in VS Code, it will produce a file called `Reconstructed.asm` and I renamed it `Reconstructed-ascii.asm`

Source Code

Disassembler.java

```
import java.io.ByteArrayOutputStream;
import java.io.DataInputStream;
import java.io.File;
import java.io.FileInputStream;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;
import java.util.Scanner;

/*
 * Disassembler for the custom MHC1 files made by Assembler.java.
 * Supports fizzbuzz.mhc-ascii and fizzbuzz.mhc-bin.
 * Run: javac Assembler.java Disassembler.java
 *      java Disassembler fizzbuzz.mhc-bin
 *   or java Disassembler fizzbuzz.mhc-ascii
 * Writes Reconstructed.asm and checks a byte-for-byte round trip.
 * Byte order: big-endian. Delayed branching must be OFF in MARS.
 */
public class Disassembler {
    static final int TEXT_BASE = 0x00400000;
    static final int DATA_BASE = 0x10010000;
    static final String[] REG = {
        "$zero", "$at", "$v0", "$v1", "$a0", "$a1", "$a2", "$a3",
        "$t0", "$t1", "$t2", "$t3", "$t4", "$t5", "$t6", "$t7",
        "$s0", "$s1", "$s2", "$s3", "$s4", "$s5", "$s6", "$s7",
        "$t8", "$t9", "$k0", "$k1", "$gp", "$sp", "$fp", "$ra"
    };

    static class Sections {
        byte[] text, data;
        Sections(byte[] t, byte[] d) { text = t; data = d; }
    }

    public static void main(String[] args) {
        if (args.length > 1) {
            System.err.println("Usage: java Disassembler [fizzbuzz.mhc-bin |
fizzbuzz.mhc-ascii]");
            return;
        }
        String input = args.length == 0 ? "fizzbuzz.mhc-bin" : args[0];
        try {
            Sections original;
            if (input.endsWith(".mhc-bin")) {
                original = readBinary(input);
            } else if (input.endsWith(".mhc-ascii")) {
                original = readAscii(input);
```

```
            } else {
                throw new IllegalArgumentException("Input must end in .mhc-bin or .mhc-
ascii");
            }

            String assembly = disassemble(original);
            // Verify by assembling the reconstructed source with the Part 3 assembler.
            Assembler.Result rebuilt = Assembler.assemble(assembly);
            if (!Arrays.equals(original.text, rebuilt.text) ||
                !Arrays.equals(original.data, rebuilt.data)) {
                throw new IllegalArgumentException("Verification FAILED: assembled bytes
do not match input.");
            }

            Files.write(Paths.get("Reconstructed.asm"),
assembly.getBytes(StandardCharsets.UTF_8));
            System.out.println("Created Reconstructed.asm");
            System.out.println("Input: " + input);
            System.out.println("Instructions: " + (original.text.length / 4));
            System.out.println("Data bytes: " + original.data.length);
            System.out.println("Verification PASSED: reassembled text and data bytes
match exactly.");
        } catch (IOException | IllegalArgumentException e) {
            System.err.println("Disassembly failed: " + e.getMessage());
        }
    }

    // Read the MHC1 binary header, then instructions and data as raw bytes.
    static Sections readBinary(String filename) throws IOException {
        File file = new File(filename);
        if (!file.isFile()) throw new IOException("Cannot find " + filename + " in " +
System.getProperty("user.dir"));
        if (file.length() < 20) throw new IOException("Binary file is too short for the
MHC1 header.");
        try (DataInputStream in = new DataInputStream(new FileInputStream(file))) {
            byte[] magic = new byte[4];
            in.readFully(magic);
            if (!Arrays.equals(magic, "MHC1".getBytes(StandardCharsets.US_ASCII)))
                throw new IllegalArgumentException("Invalid MHC1 file header.");
            int textBase = in.readInt();
            int textSize = in.readInt();
            int dataBase = in.readInt();
            int dataSize = in.readInt();
            if (textBase != TEXT_BASE || dataBase != DATA_BASE)
                throw new IllegalArgumentException("Unexpected .text or .data starting
address.");
            if (textSize < 0 || textSize % 4 != 0 || dataSize < 0 ||
                (long) textSize + dataSize + 20 != file.length())
                throw new IllegalArgumentException("Invalid section sizes in the binary
header.");
            byte[] text = new byte[textSize];
            byte[] data = new byte[dataSize];
            in.readFully(text);
            in.readFully(data);
            return new Sections(text, data);
        }
    }

    // Read the text format from Assembler.writeAscii(): 32 bits for code,
    // one 8-bit line per data byte. The section lines distinguish the two.
    static Sections readAscii(String filename) throws IOException {
```

```
        File file = new File(filename);
        if (!file.isFile()) throw new IOException("Cannot find " + filename + " in " +
System.getProperty("user.dir"));
        ByteArrayOutputStream text = new ByteArrayOutputStream();
        ByteArrayOutputStream data = new ByteArrayOutputStream();
        int section = 0, textSize = -1, dataSize = -1, lineNo = 0;
        try (Scanner scanner = new Scanner(file, "UTF-8")) {
            while (scanner.hasNextLine()) {
                lineNo++;
                String line = scanner.nextLine().trim();
                if (line.isEmpty()) continue;
                if (line.startsWith("# .text base=")) {
                    if (section != 0) throw new
IllegalArgumentException("Unexpected .text section at line " + lineNo);
                    String[] parts = line.split("\\s+");
                    if (parts.length != 4 || !parts[2].equals("base=0x00400000"))
                        throw new IllegalArgumentException("Invalid .text header at line
" + lineNo);
                    textSize = readSize(parts[3], lineNo);
                    section = 1;
                } else if (line.startsWith("# .data base=")) {
                    if (section != 1) throw new
IllegalArgumentException("Unexpected .data section at line " + lineNo);
                    String[] parts = line.split("\\s+");
                    if (parts.length != 4 || !parts[2].equals("base=0x10010000"))
                        throw new IllegalArgumentException("Invalid .data header at line
" + lineNo);
                    dataSize = readSize(parts[3], lineNo);
                    section = 2;
                } else if (line.startsWith("#")) {
                    continue;
                } else if (section == 1 && line.matches("[01]{32}")) {
                    for (int i = 0; i < 32; i += 8)
text.write(Integer.parseInt(line.substring(i, i + 8), 2));
                } else if (section == 2 && line.matches("[01]{8}")) {
                    data.write(Integer.parseInt(line, 2));
                } else {
                    throw new IllegalArgumentException("Invalid binary digits or section
at line " + lineNo + ": " + line);
                }
            }
        }
        if (section != 2 || textSize != text.size() || dataSize != data.size())
            throw new IllegalArgumentException("Missing section or byte counts do not
match the ASCII header.");
        return new Sections(text.toByteArray(), data.toByteArray());
    }

    static int readSize(String token, int line) {
        if (!token.matches("bytes=[0-9]+"))
            throw new IllegalArgumentException("Invalid byte count on line " + line);
        return Integer.parseInt(token.substring(6));
    }

    // Convert each 32-bit word to a real MIPS instruction. Create new label
    // names for branch/jump targets, since original names are not in the bytes.
    static String disassemble(Sections file) {
        int count = file.text.length / 4;
        int[] words = new int[count];
        for (int i = 0; i < count; i++) {
            int p = i * 4;
```

```
            words[i] = ((file.text[p] & 255) << 24) |
                       ((file.text[p + 1] & 255) << 16) |
                       ((file.text[p + 2] & 255) << 8) |
                       (file.text[p + 3] & 255);
        }
        Map<Integer, String> labels = new TreeMap<>();
        for (int i = 0; i < count; i++) {
            int word = words[i], op = word >>> 26;
            int pc = TEXT_BASE + i * 4;
            int target = 0;
            if (op == 4 || op == 5) {
                target = pc + 4 + ((short) (word & 65535) * 4);
            } else if (op == 2 || op == 3) {
                target = ((pc + 4) & 0xF0000000) | ((word & 0x03FFFFFF) << 2);
            } else {
                continue;
            }
            if (target < TEXT_BASE || target >= TEXT_BASE + file.text.length || ((target
- TEXT_BASE) % 4 != 0))
                throw new IllegalArgumentException("Branch/jump at 0x" + hex(pc) + "
targets outside .text: 0x" + hex(target));
            labels.put(target, "L_" + hex(target));
        }

        StringBuilder output = new StringBuilder();
        if (file.data.length > 0) {
            output.append(".data\n");
            int start = 0;
            while (start < file.data.length) {
                int end = start;
                while (end < file.data.length && file.data[end] != 0) end++;
                if (end == file.data.length)
                    throw new IllegalArgumentException("Data at offset " + start + " is
missing the .asciiz zero terminator.");
                output.append("data_").append(start).append(": .asciiz \"");
                for (int k = start; k < end; k++) {
                    int c = file.data[k] & 255;
                    if (c == '\n') output.append("\\n");
                    else if (c == '\r') output.append("\\r");
                    else if (c == '\t') output.append("\\t");
                    else if (c == '"') output.append("\\\"");
                    else if (c == '\\') output.append("\\\\");
                    else if (c >= 32 && c <= 126) output.append((char) c);
                    else throw new IllegalArgumentException("Unsupported data byte 0x" +
                            Integer.toHexString(c) + " at data offset " + k);
                }
                output.append("\"\n");
                start = end + 1;
            }
        }
        output.append(".text\n");
        for (int i = 0; i < count; i++) {
            int pc = TEXT_BASE + i * 4;
            if (labels.containsKey(pc)) output.append(labels.get(pc)).append(":\n");
            output.append("    ").append(decode(words[i], pc, labels)).append('\n');
        }
        return output.toString();
    }

    static String decode(int w, int pc, Map<Integer, String> labels) {
        int op = w >>> 26;
```

```
        int rs = (w >>> 21) & 31, rt = (w >>> 16) & 31;
        int rd = (w >>> 11) & 31, shamt = (w >>> 6) & 31;
        int funct = w & 63;
        int imm = (short) w;
        switch (op) {
            case 0:
                if (w == 12) return "syscall";
                if (funct == 0x21 && shamt == 0) return "addu " + REG[rd] + ", " +
REG[rs] + ", " + REG[rt];
                if (funct == 0x1A && rd == 0 && shamt == 0) return "div " + REG[rs] + ",
" + REG[rt];
                if (funct == 0x10 && rs == 0 && rt == 0 && shamt == 0) return "mfhi " +
REG[rd];
                if (funct == 0 && rs == 0) return "sll " + REG[rd] + ", " + REG[rt] + ",
" + shamt;
                if (funct == 8 && rt == 0 && rd == 0 && shamt == 0) return "jr " +
REG[rs];
                break;
            case 8: return "addi " + REG[rt] + ", " + REG[rs] + ", " + imm;
            case 9: return "addiu " + REG[rt] + ", " + REG[rs] + ", " + imm;
            case 13: return "ori " + REG[rt] + ", " + REG[rs] + ", " + (w & 65535);
            case 15:
                if (rs == 0) return "lui " + REG[rt] + ", " + (w & 65535);
                break;
            case 4:
            case 5:
                int branch = pc + 4 + imm * 4;
                return (op == 4 ? "beq " : "bne ") + REG[rs] + ", " + REG[rt] + ", " +
labels.get(branch);
            case 2:
            case 3:
                int jump = ((pc + 4) & 0xF0000000) | ((w & 0x03FFFFFF) << 2);
                return (op == 2 ? "j " : "jal ") + labels.get(jump);
            case 35: return "lw " + REG[rt] + ", " + imm + "(" + REG[rs] + ")";
            case 43: return "sw " + REG[rt] + ", " + imm + "(" + REG[rs] + ")";
        }
        throw new IllegalArgumentException("Unsupported MIPS word 0x" + hex(w) + " at 0x"
+ hex(pc));
    }

    static String hex(int n) { return String.format("%08X", n); }
}
```

Output

The program will disassemble the machine code and produce a file called `Reconstructed.asm`.
I first run `java Disassembler fizzbuzz.mhc-ascii` in VS Code, it will produce a file called `Reconstructed.asm` and I renamed it `Reconstructed-ascii.asm`
Then I run `java Disassembler fizzbuzz.mhc-bin` in VS Code and I renamed `Reconstructed.asm` to `Reconstructed-bin.asm`

Reconstructed-ascii.asm

```
.data
data_0: .asciiz "Fizz"
data_5: .asciiz "Buzz"
data_10: .asciiz "FizzBuzz"
.text
```

```
    addiu $t0, $zero, 1
    addiu $t1, $zero, 101
L_00400008:
    j L_00400018
L_0040000C:
    addi $t0, $t0, 1
    bne $t0, $t1, L_00400008
    j L_004000AC
L_00400018:
    addiu $at, $zero, 3
    div $t0, $at
    mfhi $t2
    addiu $at, $zero, 5
    div $t0, $at
    mfhi $t3
    beq $t2, $zero, L_0040003C
    beq $t3, $zero, L_0040007C
    j L_00400048
L_0040003C:
    beq $t3, $zero, L_00400094
    j L_00400064
L_00400044:
    j L_0040000C
L_00400048:
    addiu $v0, $zero, 1
    addu $a0, $t0, $zero
    syscall
L_00400054:
    addiu $v0, $zero, 11
    addiu $a0, $zero, 10
    syscall
    j L_00400044
L_00400064:
    addiu $v0, $zero, 4
    lui $a0, 4097
    ori $a0, $a0, 0
    syscall
    j L_00400078
L_00400078:
    j L_00400054
L_0040007C:
    addiu $v0, $zero, 4
    lui $a0, 4097
    ori $a0, $a0, 5
    syscall
    j L_00400090
L_00400090:
    j L_00400054
L_00400094:
    addiu $v0, $zero, 4
    lui $a0, 4097
    ori $a0, $a0, 10
    syscall
    j L_004000A8
L_004000A8:
    j L_00400054
L_004000AC:
    addiu $v0, $zero, 10
    syscall
```

Output for Reconstructed-ascii.asm

```
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
16
17
Fizz
19
Buzz
Fizz
22
23
Fizz
Buzz
26
Fizz
28
29
FizzBuzz
31
32
Fizz
34
Buzz
Fizz
37
38
Fizz
Buzz
41
Fizz
43
44
FizzBuzz
46
47
Fizz
49
Buzz
Fizz
52
53
Fizz
Buzz
56
Fizz
58
59
FizzBuzz
61
```

```
62
Fizz
64
Buzz
Fizz
67
68
Fizz
Buzz
71
Fizz
73
74
FizzBuzz
76
77
Fizz
79
Buzz
Fizz
82
83
Fizz
Buzz
86
Fizz
88
89
FizzBuzz
91
92
Fizz
94
Buzz
Fizz
97
98
Fizz
Buzz

-- program is finished running --
```

Reconstructed-bin.asm

Then I run `java Disassembler fizzbuzz.mhc-bin` in VS Code and I renamed `Reconstructed.asm` to `Reconstructed-bin.asm`

```
.data
data_0: .asciiz "Fizz"
data_5: .asciiz "Buzz"
data_10: .asciiz "FizzBuzz"
.text
    addiu $t0, $zero, 1
    addiu $t1, $zero, 101
L_00400008:
    j L_00400018
L_0040000C:
    addi $t0, $t0, 1
    bne $t0, $t1, L_00400008
    j L_004000AC
L_00400018:
```

```
    addiu $at, $zero, 3
    div $t0, $at
    mfhi $t2
    addiu $at, $zero, 5
    div $t0, $at
    mfhi $t3
    beq $t2, $zero, L_0040003C
    beq $t3, $zero, L_0040007C
    j L_00400048
L_0040003C:
    beq $t3, $zero, L_00400094
    j L_00400064
L_00400044:
    j L_0040000C
L_00400048:
    addiu $v0, $zero, 1
    addu $a0, $t0, $zero
    syscall
L_00400054:
    addiu $v0, $zero, 11
    addiu $a0, $zero, 10
    syscall
    j L_00400044
L_00400064:
    addiu $v0, $zero, 4
    lui $a0, 4097
    ori $a0, $a0, 0
    syscall
    j L_00400078
L_00400078:
    j L_00400054
L_0040007C:
    addiu $v0, $zero, 4
    lui $a0, 4097
    ori $a0, $a0, 5
    syscall
    j L_00400090
L_00400090:
    j L_00400054
L_00400094:
    addiu $v0, $zero, 4
    lui $a0, 4097
    ori $a0, $a0, 10
    syscall
    j L_004000A8
L_004000A8:
    j L_00400054
L_004000AC:
    addiu $v0, $zero, 10
    syscall
```

Output for Reconstructed-bin.asm

```
1
2
Fizz
4
Buzz
Fizz
7
8
```

```
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
16
17
Fizz
19
Buzz
Fizz
22
23
Fizz
Buzz
26
Fizz
28
29
FizzBuzz
31
32
Fizz
34
Buzz
Fizz
37
38
Fizz
Buzz
41
Fizz
43
44
FizzBuzz
46
47
Fizz
49
Buzz
Fizz
52
53
Fizz
Buzz
56
Fizz
58
59
FizzBuzz
61
62
Fizz
64
Buzz
Fizz
67
68
Fizz
```

```
Buzz
71
Fizz
73
74
FizzBuzz
76
77
Fizz
79
Buzz
Fizz
82
83
Fizz
Buzz
86
Fizz
88
89
FizzBuzz
91
92
Fizz
94
Buzz
Fizz
97
98
Fizz
Buzz

-- program is finished running --
```

Compare to the Output of Woo_Matthew_Old_FizzBuzz.asm

It has the same output as `Reconstructed-ascii.asm` and `Reconstructed-bin.asm`

```
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
16
17
Fizz
19
Buzz
Fizz
22
23
```

```
Fizz
Buzz
26
Fizz
28
29
FizzBuzz
31
32
Fizz
34
Buzz
Fizz
37
38
Fizz
Buzz
41
Fizz
43
44
FizzBuzz
46
47
Fizz
49
Buzz
Fizz
52
53
Fizz
Buzz
56
Fizz
58
59
FizzBuzz
61
62
Fizz
64
Buzz
Fizz
67
68
Fizz
Buzz
71
Fizz
73
74
FizzBuzz
76
77
Fizz
79
Buzz
Fizz
82
83
Fizz
```

```
Buzz
86
Fizz
88
89
FizzBuzz
91
92
Fizz
94
Buzz
Fizz
97
98
Fizz
Buzz

-- program is finished running --
```